



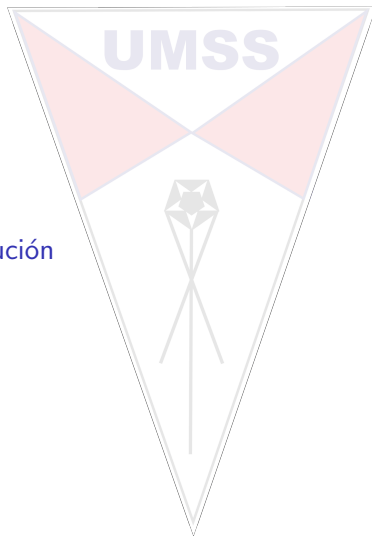
Contenido

Antecedentes

Método de sustitución

Teorema Maestro

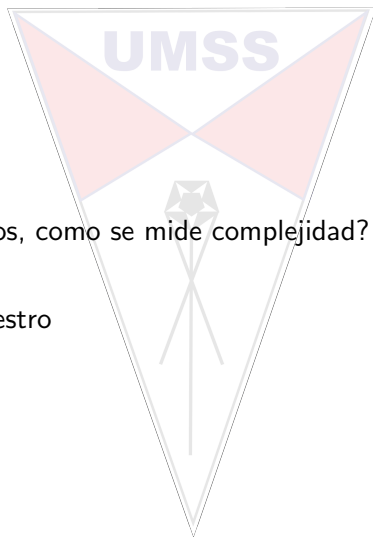
Ejercicios



Antecedentes

Procesos recursivos, como se mide complejidad?

- ▶ Sustitución
- ▶ Teorema maestro



Método de sustitución

UMSS

Reemplazar las llamadas recursivas por su tiempo de ejecución

Forma general

$$proc(n) = proc(n')$$

$$T_{proc}(n) = T_{proc}(n')$$

Método de sustitución

UMSS

$$\text{proc}(n) = \begin{cases} x & \text{si } n \leq 1 \\ \text{proc}(n-1) + y & \text{si } n > 1 \end{cases}$$

$$T_{\text{proc}}(n) = \begin{cases} c1 & \text{si } n \leq 1 \\ T_{\text{proc}}(n-1) + c2 & \text{si } n > 1 \end{cases}$$

Método de sustitución

$$T_{proc}(n) = \begin{cases} c1 & \text{si } n \leq 1 \quad \wedge \quad c1 = \Theta(1) \\ T_{proc}(n-1) + c2 & \text{si } n > 1 \end{cases}$$

$$T_{proc}(n) \equiv \underbrace{T_{proc}(n-1)} + c2 \quad (1)$$

$$\equiv \underbrace{T_{proc}(n-2)} + 2 * c2 \quad (2)$$

$$\equiv \underbrace{T_{proc}(n-3)} + 3 * c2 \quad (3)$$

$$\equiv \underbrace{T_{proc}(n-4)} + 4 * c2 \quad (4)$$

$$\dots\dots\dots \quad (5)$$

$$\equiv \underbrace{T_{proc}(n - (n-1))} + (n-1) * c2 \quad (6)$$

$$\equiv c1 + (n-1) * c2 \quad (7)$$

$$\equiv c1 + n * c2 - c2 \quad (8)$$

Por lo que:

$$T_{proc}(n) = O(n * c2) \quad (9)$$

Método de sustitución

$$\text{proc}(n) = \begin{cases} x & \text{si } n \leq 1 \\ \text{proc}(n-1) + \text{proc}(n-1) + y & \text{si } n > 1 \end{cases}$$

$$T_{\text{proc}}(n) = \begin{cases} c1 & \text{si } n \leq 1 \\ 2 * T_{\text{proc}}(n-1) + c2 & \text{si } n > 1 \end{cases}$$

Método de sustitución

$$T_{proc}(n) = \begin{cases} c1 & \text{si } n \leq 1 \\ 2 * T_{proc}(n-1) + c2 & \text{si } n > 1 \end{cases} \quad \wedge \quad c1 = \Theta(1)$$

$$T_{proc}(n) \equiv 2 \underbrace{T_{proc}(n-1)} + c2 \quad (1)$$

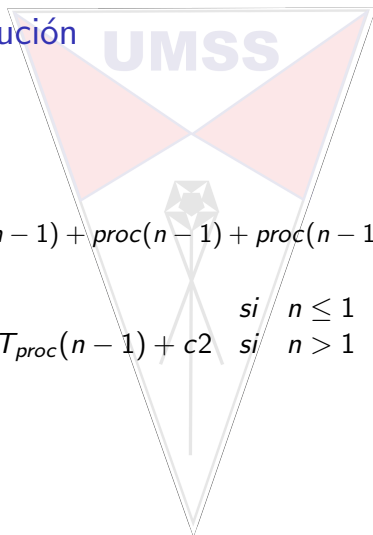
$$\equiv 2^2 \underbrace{T_{proc}(n-2)} + (2^2 - 1)c2 \quad (2)$$

Completa la sustitución....

Y demuestra que:

$$T_{proc}(n) = O(2^n * c2) \quad (3)$$

Método de sustitución



$$\text{proc}(n) = \begin{cases} x & \text{si } n \leq 1 \\ \text{proc}(n-1) + \text{proc}(n-1) + \text{proc}(n-1) + y & \text{si } n > 1 \end{cases}$$

$$T_{\text{proc}}(n) = \begin{cases} c1 & \text{si } n \leq 1 \\ 3 * T_{\text{proc}}(n-1) + c2 & \text{si } n > 1 \end{cases}$$

Método de sustitución

$$T_{proc}(n) = \begin{cases} c1 & \text{si } n \leq 1 \\ 3 * T_{proc}(n-1) + c2 & \text{si } n > 1 \end{cases} \quad \wedge \quad c1 = \Theta(1)$$

$$T_{proc}(n) \equiv 3 \underbrace{T_{proc}(n-1)} + c2 \quad (1)$$

$$\equiv 3^2 \underbrace{T_{proc}(n-2)} + (3+1)c2 \quad (2)$$

Completa la sustitución....

Y demuestra que:

$$T_{proc}(n) = O(3^n * c2) \quad (3)$$

Método de sustitución

$$\text{proc}(n) = \begin{cases} x & \text{si } n \leq 1 \\ \underbrace{\text{proc}(n-1) + \dots + \text{proc}(n-1)}_{a_n} + y & \text{si } n > 1 \end{cases}$$

$$T_{\text{proc}}(n) = \begin{cases} c1 & \text{si } n \leq 1 \\ a_n * T_{\text{proc}}(n-1) + c2 & \text{si } n > 1 \end{cases}$$

Método de sustitución

$$T_{proc}(n) = \begin{cases} c1 & \text{si } n \leq 1 \\ a_n * T_{proc}(n-1) + c2 & \text{si } n > 1 \end{cases} \quad \wedge \quad c1 = \Theta(1)$$

$$T_{proc}(n) \equiv a_n \underbrace{T_{proc}(n-1)} + c2 \quad (1)$$

$$\equiv a_n^2 \underbrace{T_{proc}(n-2)} + (a_n + 1)c2 \quad (2)$$

Completa la sustitución....

Y demuestra que:

$$T_{proc}(n) = O(a_n^n * c2) \quad (3)$$

Teorema Maestro

Para valores $a \geq 1 \wedge b \geq 1$

$$\text{proc}(n) = \begin{cases} x & \text{si } n \leq 1 \\ \underbrace{\text{proc}\left(\frac{n}{b}\right) + \dots + \text{proc}\left(\frac{n}{b}\right)}_a + y & \text{si } n > 1 \end{cases}$$

$$T_{\text{proc}}(n) = \begin{cases} \Theta(1) & \text{si } n \leq 1 \\ a * T_{\text{proc}}\left(\frac{n}{b}\right) + f(n) & \text{si } n > 1 \end{cases}$$

Teorema Maestro - Ejemplo

$$T_{proc}(n) = \begin{cases} \Theta(1) & \text{si } n \leq 1 \\ 2 * T_{proc}(\frac{n}{2}) + f(n) & \text{si } n > 1 \end{cases}$$

$$T_{proc}(n) \equiv \underbrace{2 T_{proc}(\frac{n}{2})} + f(n) \quad (1)$$

$$\equiv \underbrace{2^2 T_{proc}(\frac{n}{2^2})} + (2^2 - 1)f(n) \quad (2)$$

Completa la sustitución....

Teorema Maestro - Ejemplo

$$T_{proc}(n) = \begin{cases} \Theta(1) & \text{si } n \leq 1 \\ 2 * T_{proc}(\frac{n}{2}) + f(n) & \text{si } n > 1 \end{cases}$$

Y demuestra que:

$$T_{proc}(n) = O(2^k * f(n)) = O(2^{\log_2 n} * f(n)) = O(n * f(n)) \quad (3)$$

$$\text{Dado que, } \frac{n}{2^k} = 1 \Rightarrow n = 2^k \Rightarrow \log_2 n = k$$

Teorema Maestro - General

$$T_{proc}(n) = \begin{cases} \Theta(1) & \text{si } n \leq 1 \\ a * T_{proc}(\frac{n}{b}) + f(n) & \text{si } n > 1 \end{cases}$$

Demuestra que $T(n) = a^{\log_b n} + f(n) * \frac{1-a^{\log_b n}}{1-a}$

Propiedad: $a^{\log_b n} = n^{\log_b a}$

Teorema Maestro - General

Caso 1 Si $f(n) \equiv O(n^c); c < \log_b a$
 $\Rightarrow T(n) = \Theta(n^{\log_b a})$

Caso 2 Si $f(n) \equiv O(n^c * \log^k n); c = \log_b a; k > 0$
 $\Rightarrow T(n) = \Theta(n^{\log_b a} * \log^{k+1} n)$
 $\Rightarrow T(n) = \Theta(n^1 * \log^1 n)$
 $\Rightarrow T(n) = \Theta(n * \log n)$

Caso 3 Si
 $f(n) \equiv \Omega(n^c) \wedge a * f(\frac{n}{b}) \leq k * f(n); c > \log_b a; k > 1$
 $\Rightarrow T(n) = \Theta(f(n))$

Complejidad

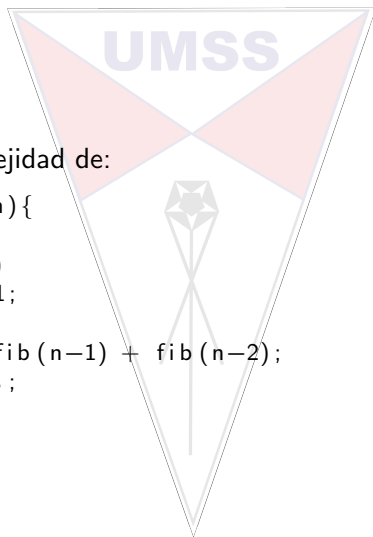
Calcular la complejidad de:

```
1  boolean buscar(int[] a, int i, int n, int dato){
2      boolean res;
3      if(i == n)
4          res = false;
5      else
6          if(a[i] == dato)
7              res = true;
8          else
9              res = buscar(a, i+1, n, dato);
10     return res;
11 }
```

Complejidad

Calcular la complejidad de:

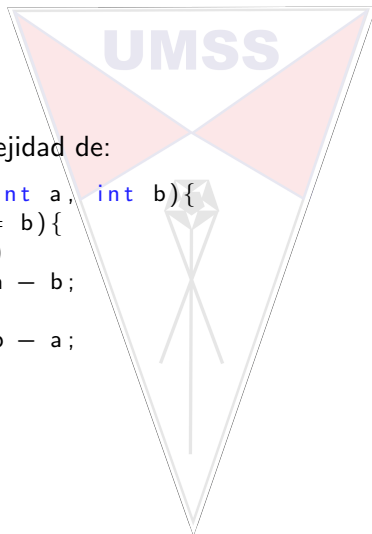
```
1  int fib(int n){  
2      int res;  
3      if(n <= 2)  
4          res = 1;  
5      else  
6          res = fib(n-1) + fib(n-2);  
7      return res;  
8  }
```



Complejidad

Calcular la complejidad de:

```
1  int proceso(int a, int b){  
2      while(a != b){  
3          if(a>b)  
4              a = a - b;  
5          else  
6              b = b - a;  
7      }  
8      return a;  
9  }
```



Complejidad

Calcular la complejidad de:

```
1  int proceso(int n){
2      int res,x,i;
3      if(n <= 1)
4          res = 1;
5      else{
6          for(i = 1; i <= n; i++){
7              x = 1;
8              while(x<n){
9                  x = x*2;
10             }
11         }
12         res = proceso(n/2) + proceso(n/2);
13     }
14     return res;
15 }
```

Referencias



THOMAS H. CORMEN CHARLES E. LEISERSON RONALD L. RIVEST
CLIFFORD STEIN (2009), *INTRODUCTION TO ALGORITHMS THIRD
EDITION*, The MIT Press